

Verxeon Technologies

Embeddable Configurable Chat Widget Case Study & Individual Intern Task Plan A Full-Stack, Multi-Tenant Chat Widget Any Business Can Drop Into Their Website

1. Case Background

Imagine you have been hired as a **Full-Stack Developer** by **Verxeon Technologies** to build **PulseChat** a new product the company wants to launch: an embeddable, configurable chat widget that any business can drop into their website with a single script tag, fully branded and backed by their own knowledge base, with no code changes required on their side.

Nothing exists yet. There is no admin dashboard, no widget script, and no backend. Your responsibility is to build **the entire product**, front to back: the admin dashboard where a business configures its bot, the tiny embeddable widget script that renders the chat bubble on someone else's site, and the Supabase backend that powers both. You will design the screens, build the widget, and connect every piece to a live, multi-tenant database. That is the whole scope. One product, one Supabase project, every feature built and made real.

The solution should be **reusable and multi-tenant from day one**. A single deployment of PulseChat must be able to serve many different businesses at once, each with its own isolated bot configuration, knowledge base, and conversation history never leaking one business's data or chats into another's.

Business Problem:

Small and mid-size businesses want an AI chat assistant on their site but cannot afford to build one, and existing tools like Intercom or Chatbase are either too expensive or too generic. They need something simple: paste one script tag, upload a few documents, pick a colour, and get a working assistant that actually answers questions about their business not a canned FAQ bot. Every one of these is a missing piece a configurable bot, an isolated widget, a scoped knowledge base, a multi-tenant chat engine that you are here to build.

Your Objective:

Design, build, test, and document a small production-quality full-stack application that delivers a working embeddable chat widget product: real dashboard metrics, per-bot branding and configuration, a knowledge base per bot that powers grounded answers, an embeddable widget script that renders cleanly on any host site, conversation logging, staff roles, and reports all built on top of a properly modelled, multi-tenant Supabase database with saved SQL scripts.

What you will learn from this case:

- How a real multi-tenant SaaS product isolates each customer's data, configuration, and usage from every other customer on the same system.
- How to build a small, embeddable widget script that renders safely inside any host page without clashing with that page's styles or blocking its load.
- How to model a relational schema in Supabase (Postgres) and write the SQL that creates it.
- How to wire a dashboard and a widget to a shared database: auth, per-bot RAG, streaming chat, and row-level security.
- How to keep every SQL script version-controlled in the codebase so the build is reproducible.

2. Solution Architecture

The build is deliberately focused: **one admin dashboard, one embeddable widget script, one Supabase backend, fully multi-tenant**. The Supabase Postgres database is the single source of truth for every account, bot, knowledge source, conversation, and message. Supabase Auth secures the dashboard; the widget itself needs no visitor login. Everything the dashboard shows bot config, knowledge base, conversation history, analytics is read live from the database, and every visitor message flows through the same backend, scoped to the correct bot.

Layer	Component
Admin dashboard	Where a business configures its bot's branding, persona, and knowledge base, built for PulseChat
Widget script	A small, framework-free script (<code>widget.js</code>) loaded via a single <code><script></code> tag, rendered in an isolated container so it never clashes with the host site's CSS
Database	Supabase Postgres tables for accounts, bots, knowledge sources, conversations, messages, staff, roles; the source of truth
Auth	Supabase Auth business/staff login to the dashboard only
RAG per bot	Each bot's knowledge base is embedded and searched independently, so answers are grounded only in that bot's own content
SQL scripts	Every table, policy, and seed saved as <code>.sql</code> files committed in the repo
Security	Row-level security and per-bot scoping ensure one account can never read another account's bots, knowledge, or conversations

Product-journey flow: a business **signs up and logs in** through Supabase Auth → the **Dashboard** reads their live bots and metrics from the database → they **create a bot**, configuring its brand colour, avatar, welcome message, and persona instructions → they **upload knowledge** (documents or URLs) that is embedded and scoped to that bot only → they **copy the embed snippet** and paste it into their own site → a visitor on that site **opens the widget** and chats → the

backend retrieves that bot's own knowledge, generates a grounded reply, and **streams it back** → every **conversation and message is logged** against the correct bot and account → an admin manages **staff roles and permissions** → they open **reports** that aggregate the real data.

Each screen and every widget request is driven by live queries against the Supabase tables, scoped by bot and account, which act as the source of truth and the isolation boundary for the whole product exactly how a real multi-tenant SaaS backend is wired.

3. Subsystem Reference

3.1 Accounts & bot configuration (Dashboard, Bots, Knowledge Base)

This subsystem guarantees that every business's bot, branding, and knowledge base are real, isolated, and easy to configure.

- **Live dashboard:** total bots, conversations this week, messages sent, and recent activity every figure read from the database, never hard-coded.
- **Bot management:** create, edit, and delete bots; configure brand colour, avatar, bot name, welcome message, and widget position, with a live preview pane.
- **Persona & instructions:** set custom system instructions per bot (e.g. "You are a support agent for Acme Inc, be friendly and concise").
- **Knowledge base:** upload documents or URLs per bot; content is embedded and scoped so a bot only ever answers from its own knowledge.
- **Embed snippet:** generate a copy-paste `<script>` tag containing that bot's unique ID for the business to paste into their own site.

3.2 Widget & conversations (Embeddable Widget, Chat Engine, Staff, Reports)

This subsystem renders the widget on any site, powers grounded replies, and reports on all of it.

- **Embeddable widget:** a small script that fetches its bot's configuration by ID, renders an isolated chat bubble and window, and never breaks the host page's styling or load time.
- **Chat engine:** on each visitor message, retrieve that bot's relevant knowledge, generate a reply grounded in it, and stream the response back to the widget.
- **Conversation logging:** persist every conversation and message, scoped to the correct bot and account, viewable from the dashboard.
- **Rate limiting:** cap requests per bot/domain so one business's widget cannot be abused and run up costs for the whole platform.
- **Staff, roles & permissions:** manage users, assign roles, and enforce permissions with row-level security.
- **Reports:** conversations per bot, messages per week, most-asked questions, and an account-usage report, each aggregated live from the tables.
- **Settings:** account information, role management, and the logged-in user's profile.

4. Individual Delivery Plan

Every intern completes **this entire plan individually** you each build your own complete product from scratch so you personally experience and master every part. Do not split the work between interns. Tasks are ordered by dependency **finish earlier tasks before starting later ones**.

Theme	Outcome
Setup & schema	Dashboard scaffolded, Supabase project live, multi-tenant schema created via SQL scripts.
Bots & dashboard	Bot CRUD with live preview works; the dashboard shows live metrics.
Knowledge base & widget	Knowledge upload works; the embeddable widget script renders on a test page.
Chat engine & staff	The widget produces grounded, streamed replies; staff roles/permissions are functional.
Reports, settings & demo	Reports aggregate live data, settings work, and a full demo is recorded.

4.1 Setup & Schema

T1.1 Scaffold the dashboard & set up Supabase *Objective:* a running admin dashboard connected to a live Supabase project.

- Scaffold a new frontend project (e.g. Next.js) and name your build (e.g. PulseChat-yourname).
- Point your project at the team repository <https://github.com/verxeon-ai/PulseChat.git> and create your own branch to work in (see Section 7 for the Git steps).
- Create a free Supabase account and a new project; save the project URL and keys in a local `.env`.
- Connect the dashboard to Supabase with the client SDK and confirm the app boots without errors.
- Build the sidebar and navigation to the product's sections: Dashboard, Bots, Knowledge Base, Conversations, Staff, Reports, Settings.

Definition of Done: the dashboard runs, connects to Supabase, shows the navigation, and your work is committed on

your own branch.

T1.2 Design & create the multi-tenant database schema *Objective:* a relational schema that keeps every account's data fully isolated, created from saved SQL.

- Design tables: `accounts`, `bots`, `knowledge_sources`, `conversations`, `messages`, `staff`, `roles`, and supporting link tables.
- Write the `CREATE TABLE` SQL with keys and foreign-key relationships (e.g. a bot references an account; a conversation references a bot).
- **Save every SQL script** into a `/supabase/sql` folder in the repo so the schema is reproducible.
- Add a seed script that inserts a couple of sample accounts, bots, and knowledge sources for testing isolation between them.

Definition of Done: the schema exists in Supabase, all SQL scripts are committed to the repo, and seed data for at least two separate accounts loads.

4.2 Bots & Dashboard

T2.1 Bot management with live preview *Objective:* a business can fully configure its bot's identity from the dashboard.

- Wire *Add*, *Edit*, and *Delete Bot* to the `bots` table.
- Build a *live preview* pane that reflects brand colour, avatar, name, welcome message, and position as they are edited.
- Store the bot's *persona/instructions* field for later use by the chat engine.
- Test the full cycle: create a bot, edit its branding, preview it, and delete it.

Definition of Done: bots can be created, configured, previewed, and deleted against real data.

T2.2 Live dashboard metrics & embed snippet *Objective:* every dashboard figure reflects the real database, and each bot has a ready-to-use embed snippet.

- Replace hard-coded cards with live queries: total bots, conversations this week, messages sent.
- Build a recent-activity feed from the latest conversations and bot changes.
- Generate a copy-paste `<script>` snippet per bot containing its unique bot ID.
- Test that creating a bot immediately produces a working, copyable embed snippet.

Definition of Done: all dashboard metrics and activity read live from the database, and every bot has a correct embed snippet.

4.3 Knowledge Base & Widget

T3.1 Knowledge base per bot *Objective:* each bot's knowledge is uploadable, stored, and scoped only to that bot.

- Wire *Upload Document* and *Add URL* to the `knowledge_sources` table, linked to a specific bot.
- Extract and store text content for each source so it can be searched at chat time.
- Add *delete* for a knowledge source.
- Test that a knowledge source uploaded to one bot never appears when querying a different bot.

Definition of Done: knowledge sources can be uploaded, stored, and deleted, and are strictly scoped to their own bot.

T3.2 Build the embeddable widget script *Objective:* a small script renders a working chat bubble on any test page, isolated from that page's styles.

- Build `widget.js`: on load, read the `data-bot-id` attribute, fetch that bot's config, and render a chat bubble in an isolated container (e.g. Shadow DOM or `iframe`).
- Ensure the script loads asynchronously and does not block the host page's render.
- Style the widget using the fetched bot config (colour, avatar, welcome message, position).
- Test by embedding the script on a plain HTML test page and confirming it renders correctly without any styling conflicts.

Definition of Done: the widget script renders a correctly branded, isolated chat bubble on an external test page from a single script tag.

4.4 Chat Engine & Staff

T4.1 Grounded chat engine *Objective:* the widget produces real, streamed answers grounded in the bot's own knowledge.

- On each visitor message, retrieve the relevant knowledge for that specific bot only.
- Generate a reply using the bot's persona instructions plus the retrieved knowledge, and stream it back to the widget.
- Persist every conversation and message to `conversations` and `messages`, scoped to the correct bot and account.
- Add basic *rate limiting* per bot/domain to prevent abuse.
- Test asking the widget a question that only one bot's knowledge base can answer, and confirm another bot cannot answer it.

Definition of Done: the widget returns grounded, streamed replies, every conversation is logged correctly, and rate limiting is in place.

T4.2 Staff, roles & permissions *Objective:* the right people can do the right things, enforced by the database.

- Wire *Users* management to the *staff* table, backed by Supabase Auth.
- Define *Roles* (e.g. owner, editor, viewer) and assign them to users within an account.
- Enforce *Permissions* with row-level security policies, saved as SQL scripts in the repo ensuring one account can never read another's bots or conversations.
- Test that a viewer cannot edit a bot's configuration while an owner can, and that a user from Account A cannot see Account B's data at all.

Definition of Done: users have roles, RLS policies enforce permissions and full account isolation, and all policy SQL is committed to the repo.

4.5 Reports, Settings & Demo

T5.1 Reports, settings & demo *Objective:* live reporting, working settings, and a polished final demo.

- Build reports that aggregate live data: conversations per bot, messages per week, and most-asked questions.
- Wire *Settings*: account information, role management, and the logged-in user's profile.
- Confirm every SQL script (schema, seed, policies) is saved in the repo and the database rebuilds from them.
- Record a 4–5 minute demo: configure a bot, embed it on a test page, and have a real conversation with it.

Definition of Done: reports and settings work against live data, all SQL is committed, and the demo is recorded.

5. Individual Task Checklist

Each intern must personally complete and be able to demonstrate **all** of the following in their own product, backed by **their own Supabase project**. Tick each item only when you can demo it working live.

#	Feature	You can demonstrate that...
1	Dashboard built & connected	the admin dashboard runs and connects to a live Supabase project.
2	Multi-tenant schema & SQL scripts	the schema is created, isolates accounts, and every SQL script is committed to the repo.
3	Live dashboard metrics	totals and activity read live from the database.
4	Bot management & preview	bots can be created, branded, previewed, and deleted against real data.
5	Embed snippet	every bot has a correct, copyable <code><script></code> embed tag.
6	Knowledge base scoping	knowledge uploaded to one bot never leaks into another bot's answers.
7	Embeddable widget	the widget script renders a branded, isolated chat bubble on an external test page.
8	Grounded chat replies	the widget answers using only that bot's own knowledge, streamed live.
9	Conversation logging	every conversation and message is persisted and viewable from the dashboard.
10	Rate limiting	requests per bot/domain are capped to prevent abuse.
11	Staff, roles & RLS	users have roles, and one account can never see another account's data.
12	Reports & settings	usage reports aggregate live data, and account/profile settings can be updated.

6. Deliverables

Each intern submits the following individually at the end of the week:

- A live, running admin dashboard and embeddable widget connected to a working, multi-tenant Supabase project with every page functional.
- Your work pushed to the team repository <https://github.com/verxeon-ai/PulseChat.git>, developed on your own branch and merged into `main` only after testing.
- A repository containing the dashboard code, the widget script, **and all SQL scripts** (schema, seed, RLS policies) under `/supabase/sql`.
- One demo recording (4–5 minutes) showing a bot configured, embedded on a test page, and answering a real question grounded in its own knowledge.

7. Database & Repository Reference

Build the dashboard, the widget script, and the backend behind both. Keep every SQL script in the repository so the entire database can be rebuilt from scratch by anyone who clones it.

Item	Detail
Team repo (push here)	https://github.com/verxeon-ai/PulseChat.git
Backend	Supabase (Postgres database and Auth)
SQL location	/supabase/sql/ every script committed to the repo
Core tables (suggested)	
accounts	business name, email, plan, created_at
bots	account_id (FK), name, brand colour, avatar URL, welcome message, position, persona instructions, status
knowledge_sources	bot_id (FK), source type (document/URL), content, uploaded_at
conversations	bot_id (FK), visitor session, started_at
messages	conversation_id (FK), role (visitor/bot), content, created_at
staff	account_id (FK), linked to Supabase Auth user, name, contact
roles	role name and permission set, joined to staff
Required SQL scripts	
schema.sql	all CREATE TABLE statements with keys and foreign keys
seed.sql	sample accounts, bots, and knowledge sources for testing isolation
policies.sql	row-level security policies enforcing account and bot-level isolation

8. Git Workflow (Simple Guide)

You will push all your work to the team repository <https://github.com/verxeon-ai/PulseChat.git>. **Never build directly on main.** Always work on your own branch, test everything, and only then merge into main. Follow these steps in order they are all you need.

Step 1 Get the code onto your machine (once):

- `git clone https://github.com/verxeon-ai/PulseChat.git`
- `cd PulseChat`

Step 2 Create your own branch and switch to it:

- `git checkout -b yourname-widget` (use your own name, e.g. ali-widget).
- This keeps your work separate from main and from other interns.

Step 3 Work, then save your progress often:

- After finishing a piece of work: `git add .`
- Save it with a clear message: `git commit -m "Add widget.js embed rendering"`
- Send it to GitHub: `git push origin yourname-widget`
- Repeat this every time you complete a task small, frequent commits are best.

Step 4 Test before you merge:

- Run the dashboard and the widget on a test page and click through every part you changed.
- Make sure nothing is broken and every feature works against Supabase.
- Only move on once you are confident your branch is fully working.

Step 5 Merge into main (only after testing):

- Open a Pull Request on GitHub from your branch into main, or ask your lead to review it.
- Once approved and tested, merge it into main.
- **Golden rule:** code only reaches main after it has been tested and works.

Handy tip: to update your branch with the latest main at any time, run `git checkout main`, then `git pull`, then `git checkout yourname-widget`, then `git merge main`.